

MODUL 1 PERCABANGAN DAN PERULANGAN

Nesa Dwi Cahyani (H1H024024)

Asisten: Muhammad Nur Bijak B

Tanggal Percobaan: 1/4/2026

TK24004-Sistem Mikrotroller Sistem

Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Abstrak

Struktur kontrol dalam pemrograman Arduino terdiri dari percabangan (*if-else*) dan perulangan (*while* dan *for*) yang berfungsi untuk mengatur alur program. Percabangan digunakan untuk pengambilan keputusan berdasarkan kondisi tertentu, sedangkan perulangan digunakan untuk menjalankan perintah secara berulang. Kedua konsep ini banyak diterapkan dalam sistem berbasis Arduino, seperti pengendalian LED dan perangkat otomatis. Kombinasi percabangan dan perulangan memungkinkan sistem bekerja secara lebih efisien, responsif, dan otomatis sesuai dengan kondisi yang diberikan.

Kata kunci: Percabangan, Perulangan, Arduino, If-Else, While, For, Struktur Kontrol.

1. PENDAHULUAN

Perkembangan teknologi mikrokontroler, khususnya Arduino, telah merancang sistem otomatis yang sederhana hingga kompleks. Dalam pemrograman Arduino, struktur kontrol memegang peranan penting karena menentukan alur kerja program dalam merespons input dan menghasilkan output. Struktur kontrol tersebut meliputi percabangan (*if-else*) dan perulangan (*while* dan *for*) yang digunakan untuk membangun logika program.

Percabangan digunakan untuk menentukan perintah yang akan dijalankan sesuai dengan kondisi yang ada, sementara perulangan memungkinkan suatu instruksi dijalankan secara berulang agar sistem dapat bekerja secara terus-menerus. Kedua konsep tersebut diterapkan melalui LED sebagai media praktik, sehingga mempermudah pemahaman mengenai bagaimana program memberikan respons terhadap kondisi yang diberikan.

2. STUDI PUSTAKA

Struktur kontrol merupakan elemen fundamental dalam pemrograman mikrokontroler untuk mengatur alur logika secara sistematis. Percabangan (*branching*) adalah struktur yang memungkinkan program untuk memilih jalur eksekusi yang berbeda berdasarkan terpenuhinya suatu kondisi logika [1]. Pada Arduino, pernyataan *if-else* digunakan untuk mengubah respon sistem secara dinamis, seperti mengatur transisi fase kecepatan kedipan LED berdasarkan variabel status yang ditentukan [2].

Sementara itu, perulangan (*looping*) didefinisikan sebagai instruksi yang mengeksekusi blok kode secara repetitif selama kondisi logika tertentu masih terpenuhi [3]. Penggunaan struktur *for* pada mikrokontroler sangat efektif untuk mengontrol deretan pin *output* secara berurutan, sehingga memungkinkan pembuatan pola gerakan LED dengan baris kode yang lebih efisien [4]. Integrasi antara percabangan dan perulangan, yang didukung oleh operator logika dasar, merupakan pondasi utama dalam pengembangan sistem kendali otomatis serta alat peraga ilmiah berbasis perangkat keras [5].

2.1 KARAKTERISTIK DAN FUNGSI LED

LED (*Light Emitting Diode*) merupakan komponen semikonduktor yang dapat memancarkan cahaya ketika arus listrik mengalir dari anoda ke katoda. Karakteristik utama LED meliputi polaritas, di mana kaki anoda harus dihubungkan ke sumber tegangan positif dan kaki katoda ke negatif, serta tegangan maju (*forward voltage*) yang diperlukan agar LED dapat menyala. LED memiliki efisiensi tinggi dalam konversi energi listrik menjadi cahaya, sehingga menghasilkan konsumsi daya yang rendah dibandingkan dengan lampu pijar.

Dalam praktikum Arduino, LED digunakan sebagai indikator visual untuk menunjukkan hasil eksekusi program atau respons mikrokontroler terhadap logika yang dijalankan. Setiap LED dapat

dinyalakan atau dimatikan melalui sinyal digital HIGH atau LOW dari pin Arduino. Penerapan LED dalam percobaan ini, seperti percabangan untuk kedipan dinamis dan perulangan untuk efek *running LED*, memungkinkan praktikan memahami hubungan langsung antara logika program dan output fisik pada perangkat keras [3], [5].

2.2 PERULANGAN *FOR* – PENJELASAN LED BERGERAK DARI KIRI KE KANAN DAN KANAN KE KIRI

Perulangan *for* digunakan dalam pemrograman Arduino untuk mengeksekusi instruksi secara berulang dengan jumlah iterasi tertentu, sehingga sangat efektif untuk mengendalikan beberapa LED secara sistematis. Pada percobaan *running LED*, perulangan *for* pertama digunakan untuk menyalakan LED dari pin 2 hingga pin 7 secara berurutan. Variabel *ledPin* dimulai dari nilai 2 dan meningkat satu per satu pada setiap iterasi hingga mencapai pin 7. Hal ini menyebabkan LED menyala secara berurutan dari kiri ke kanan, sehingga tercipta efek visual pergerakan cahaya yang terlihat kontinu [1], [2], [3].

Setelah LED menyala dari kiri ke kanan, perulangan *for* kedua digunakan untuk mengembalikan efek pergerakan dari kanan ke kiri. Pada perulangan ini, variabel *ledPin* dimulai dari nilai 7 dan berkurang satu per satu hingga mencapai pin 2. Akibatnya, LED menyala secara berurutan dari pin dengan nomor besar ke kecil, menciptakan efek bolak-balik atau *bounce effect*. Penerapan perulangan *for* ini menunjukkan bagaimana penggunaan struktur perulangan memungkinkan pengendalian output yang efisien tanpa perlu menulis perintah *digitalWrite()* untuk setiap LED secara individual [4], [5].

Dengan kombinasi kedua perulangan ini, pola *running LED* dapat dijalankan secara terus-menerus, memperlihatkan hubungan langsung antara logika program dan respons perangkat keras pada mikrokontroler Arduino [1], [2], [3], [4], [5].

2.3 IMPLIKASI PRAKTIKUM – PEMAHAMAN LOGIKA PROGRAM DAN HUBUNGAN DENGAN PERANGKAT KERAS

Praktikum Modul 1 memberikan pemahaman langsung mengenai bagaimana logika program diterapkan pada perangkat keras mikrokontroler. Dengan melakukan percobaan percabangan dan perulangan, praktikan dapat melihat secara nyata bagaimana keputusan logika (*if-else*) dan eksekusi instruksi berulang (*for* atau *while*) mempengaruhi perilaku LED. Hal ini memperlihatkan hubungan timbal balik antara kode program yang ditulis di

Arduino IDE dan respons fisik yang dihasilkan oleh perangkat keras [1], [2], [3].

Selain itu, praktikum ini menekankan pentingnya desain rangkaian elektronik yang aman dan efisien, termasuk penggunaan resistor untuk membatasi arus dan mencegah kerusakan pada LED maupun pin mikrokontroler. Pemahaman ini menjadi dasar bagi pengembangan sistem *embedded* yang lebih kompleks, di mana logika program harus disinkronkan dengan perangkat keras untuk menghasilkan sistem yang responsif dan stabil. Dengan pengalaman praktis ini, praktikan diharapkan mampu merancang dan memodifikasi program serta rangkaian elektronik secara efektif, sehingga dapat menciptakan aplikasi *embedded system* yang sesuai kebutuhan [4], [5].

3. METODOLOGI

3.1 KOMPONEN DAN ALAT

Dalam praktikum ini, digunakan perangkat keras yang terintegrasi pada breadboard untuk memudahkan proses bongkar-pasang rangkaian tanpa perlu penyolderan [1]

Tabel 3-1 Daftar Alat dan Bahan Praktikum

Nama Alat/Bahan	Jumlah	Fungsi
Arduino Uno R3	1 Unit	Unit kendali utama (mikrokontroler)
Breadboard	1 Unit	Papan penghubung rangkaian sementara
LED	7 Buah	Indikator visual output digital
Resistor 220 Ohm	7 Buah	Pembatas arus untuk melindungi LED
Kabel Jumper	Secukupnya	Penghubung elektrikal antar komponen
Kabel USB Type B	1 Buah	Komunikasi data dan catu daya dari PC

PC dengan Arduino IDE	1 Unit	Perangkat lunak pemrograman dan compiler
-----------------------------	--------	--

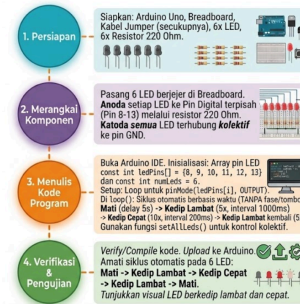
3.2 PROSEDUR PERCOBAAN 1: PERCABANGAN

Percobaan 1 dilakukan untuk menguji penggunaan logika *if-else* dalam mengatur interval waktu kedipan LED. Tahapan yang dilakukan meliputi :

1. Menyiapkan papan Arduino Uno dan menghubungkannya ke komputer.
Merangkai satu buah LED pada Pin Digital 6 dengan tambahan resistor 220 Ohm yang terhubung ke GND.
2. Menuliskan program pada Arduino IDE dengan mendeklarasikan variabel global `int timeDelay = 1000`.
3. Pada fungsi `loop()`, LED dinyalakan dan dimatikan menggunakan perintah `digitalWrite(ledPin, HIGH)` dan `LOW`, dengan jeda waktu yang diatur oleh `delay(timeDelay)`.
4. Menerapkan logika percabangan, yaitu ketika nilai `timeDelay` kurang dari atau sama dengan 100, maka nilai tersebut diatur ulang menjadi 1000 dan ditambahkan jeda selama 3000 ms. Jika kondisi tersebut tidak terpenuhi, maka `timeDelay` akan dikurangi sebesar 100 ms setiap perulangan.
5. Melakukan proses kompilasi (*verify*) dan mengunggah (*upload*) program ke Arduino.
6. Mengamati hasil berupa pola kedipan LED yang semakin cepat, kemudian berhenti sesaat, lalu kembali melambat.



Gambar 3-1 Diagram Langkah-langkah Percobaan 1



Gambar 3-2 Diagram Langkah-langkah Percobaan 2

3.3 PROSEDUR PERCOBAAN 2: PERULANGAN

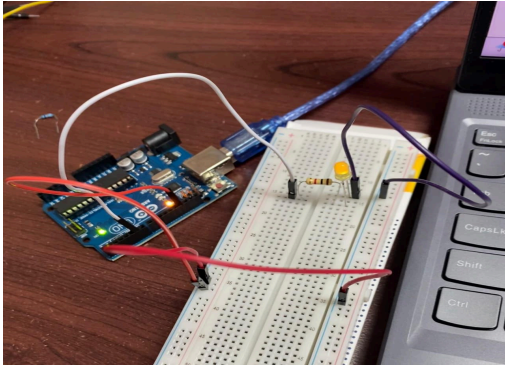
Percobaan 2 berfokus pada pemanfaatan *loop for* untuk mengontrol enam buah LED secara berurutan. Tahapan yang dilakukan adalah sebagai berikut :

1. Enam LED dipasang berjajar pada breadboard.
2. Setiap anoda LED dihubungkan ke Pin Digital 2 hingga 7 secara berurutan.
3. Seluruh katoda LED dihubungkan ke jalur negatif breadboard yang terhubung ke pin GND pada Arduino.
4. Program dituliskan dengan memanfaatkan *loop for* pada bagian `setup()` untuk mengatur pin 2 sampai 7 sebagai OUTPUT secara otomatis.
5. Pada bagian `loop()`, dibuat dua buah struktur *for*, yaitu satu untuk menyalakan LED dari pin terkecil ke terbesar (maju), dan satu lagi dari pin terbesar ke terkecil (mundur).
6. Di dalam setiap perulangan, digunakan perintah `digitalWrite(ledPin, HIGH)`, `delay(timer)`, dan `digitalWrite(ledPin, LOW)` untuk menghasilkan efek cahaya yang bergerak.
7. Hasil pengamatan menunjukkan LED menyala secara bergantian sesuai

urutan, tanpa ada yang terlewat maupun menyala di luar pola yang telah ditentukan.

4. HASIL DAN ANALISIS

4.1 ANALISIS HASIL PERCOBAAN



Gambar 4-1 Rangkaian Percobaan 1

Berdasarkan hasil pengamatan pada Percobaan 1, LED menunjukkan pola kedipan yang berubah secara dinamis. Pada awal program, LED berkedip dengan interval waktu sebesar 1000 ms. Seiring berjalannya program, interval tersebut semakin kecil sehingga kedipan LED menjadi semakin cepat. Hal ini terjadi karena pada setiap iterasi fungsi `loop()`, nilai variabel *timeDelay* dikurangi sebesar 100 melalui perintah yang berada pada blok *else* [4], [5].

Ketika nilai *timeDelay* mencapai batas tertentu, yaitu kurang dari atau sama dengan 100 ms, program akan mengevaluasi kondisi tersebut sebagai benar dan masuk ke dalam blok *if*. Pada kondisi ini, nilai *timeDelay* diatur kembali menjadi 1000 ms dan diberikan jeda tambahan selama 3000 ms. Jeda ini berfungsi sebagai pemisah antar siklus sehingga pengguna dapat membedakan perubahan kecepatan kedipan LED sebelum kembali ke kondisi awal [4].

Perubahan kecepatan kedipan LED ini menunjukkan bahwa struktur percabangan *if* berperan dalam mengatur alur eksekusi program berdasarkan kondisi tertentu. Hal ini sesuai dengan konsep bahwa percabangan digunakan untuk mengambil keputusan dalam program dengan mengevaluasi suatu kondisi logika [1], [2], [3].

4.1.1 JAWABAN PERTANYAAN PRAKTIKUM PERCOBAAN 1

1. Pada Kondisi apa program masuk ke blok *if*?

Program akan menjalankan blok *if* ketika nilai variabel *timeDelay* berada pada kondisi kurang dari atau sama dengan 100 ($timeDelay \leq 100$). Hal ini menandakan bahwa nilai *delay* telah mencapai batas minimum yang ditetapkan dalam program. Mengingat nilai awal *timeDelay* adalah 1000 dan berkurang sebesar 100 pada setiap iterasi, maka kondisi tersebut akan tercapai setelah beberapa kali perulangan hingga nilai *delay* menyentuh 100 [4], [5].

2. Pada Kondisi Apa program masuk ke blok *else*?

Sebaliknya, program akan masuk ke blok *else* ketika nilai *timeDelay* masih lebih besar dari 100 ($timeDelay > 100$). Pada kondisi ini, nilai *delay* akan terus dikurangi secara bertahap, sehingga kedipan LED menjadi semakin cepat seiring berjalannya waktu [4], [5].

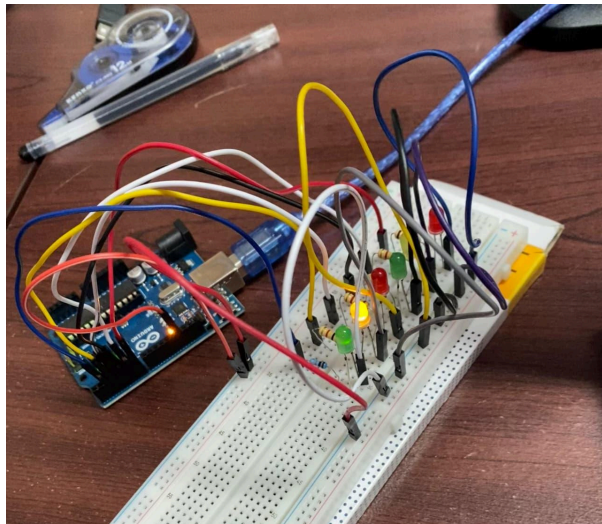
3. Apa fungsi dari perintah *delay(timeDelay)*?

Perintah *delay(timeDelay)* berfungsi untuk menunda sementara jalannya program selama waktu tertentu dalam satuan *milidetik*. Besarnya waktu tunda ini ditentukan oleh nilai variabel *timeDelay*. Dalam percobaan ini, fungsi tersebut digunakan untuk mengatur durasi LED dalam kondisi menyala maupun mati, sehingga secara langsung mempengaruhi kecepatan kedipan LED [4], [5].

4. Modifikasi Program (Lambat → Cepat → Lambat/Mati)

Untuk memodifikasi program agar pola perubahan tidak langsung kembali ke awal, melainkan berubah secara bertahap dari cepat ke lambat, diperlukan penambahan variabel baru sebagai pengatur arah perubahan *delay*. Variabel ini berfungsi untuk menentukan apakah nilai *delay* sedang dikurangi atau justru ditambah, sehingga pola kedipan dapat berubah dari lambat ke cepat, lalu kembali melambat secara bertahap [1], [2], [3].

4.2 ANALISIS HASIL PERCOBAAN 2



Gambar 4-2 Rangkaian Percobaan 2

Berdasarkan hasil percobaan 2A, LED menghasilkan pola nyala berupa *running LED* yang bergerak dari kiri ke kanan dan kemudian kembali dari kanan ke kiri secara berulang. Pola ini terbentuk melalui penggunaan struktur perulangan *for* yang mengatur urutan aktivasi pin digital pada Arduino Uno [1], [2], [3].

Pada perulangan pertama, digunakan struktur: *or (int ledPin = 2; ledPin < 8; ledPin++)*. Pada perulangan ini, variabel *ledPin* akan bernilai 2 hingga 7 secara berurutan. Setiap nilai digunakan untuk menyalakan LED yang terhubung pada pin tersebut, sehingga menghasilkan efek pergerakan dari kiri ke kanan [4], [5].

Setelah itu, digunakan perulangan kedua: *or (int ledPin = 7; ledPin >= 2; ledPin--)*. Perulangan ini berjalan dengan arah sebaliknya, yaitu dari pin 7 ke pin 2. Hal ini menyebabkan LED menyala dari kanan ke kiri, sehingga menciptakan efek bolak-balik (*bounce effect*) [4], [5].

Penggunaan struktur perulangan *for* ini menunjukkan bahwa pengendalian banyak output dapat dilakukan secara efisien tanpa harus menuliskan perintah secara berulang. Hal ini sesuai dengan konsep bahwa perulangan digunakan untuk mengeksekusi instruksi secara berulang dengan jumlah iterasi tertentu [1], [2], [3].

Tabel 4-1 Logika Sekuensial *Running LED*

Fase	Pin Aktif	Status LED	Arah Gerakan
Iterasi 1	2 → 7	Menyala berurutan	Kiri ke kanan

Iterasi 2	7 → 2	Menyala berurutan	Kanan ke kiri
4.2.1 Jawaban Pertanyaan Praktikum Percobaan 2			

1. Gambaran rangkaian schematic LED *running*
 Pada percobaan ini digunakan enam buah LED yang dihubungkan ke pin digital Arduino Uno, yaitu pin 2 hingga 7. Setiap LED disusun secara paralel dengan konfigurasi sebagai berikut:
 - Kaki anoda (positif) LED dihubungkan ke pin digital Arduino (pin 2–7).
 - Kaki katoda (negatif) LED dihubungkan ke resistor dengan nilai sekitar 220 Ω .
 - Ujung resistor lainnya dihubungkan ke ground (GND).

Konfigurasi ini bertujuan untuk memastikan setiap LED mendapatkan arus yang aman, sehingga tidak merusak LED maupun pin mikrokontroler [4], [5].

2. Penjelasan efek LED dari kiri ke kanan
 Efek LED berjalan dari kiri ke kanan dihasilkan oleh perulangan *for* dengan arah inkremental, yaitu:

```
for (int ledPin = 2; ledPin < 8; ledPin++)
```

Pada perulangan ini:

- variabel *ledPin* dimulai dari nilai 2 sebagai pin pertama
- Perulangan berjalan hingga kurang dari 8 (pin 2 sampai 7).
- Setiap iterasi menaikkan nilai *ledPin* sebesar satu (*increment*).

Akibatnya, LED menyala secara berurutan dari pin kecil ke besar sehingga menghasilkan efek pergerakan dari kiri ke kanan [4], [5].

3. Penjelasan efek LED dari kanan ke kiri
 Efek LED dari kanan ke kiri dihasilkan oleh perulangan *for* dengan arah dekremental, yaitu:

```
for (int ledPin = 7; ledPin >= 2; ledPin--)
```

Pada perulangan ini:

- variabel *ledPin* dimulai dari nilai 7 dan berkurang hingga mencapai 2.
 - Setiap iterasi menurunkan nilai *ledPin* sebesar satu (*decrement*), sehingga LED menyala dari pin besar ke kecil dan menghasilkan efek gerakan dari kanan ke kiri [4], [5].
4. Program modifikasi: 3 LED kiri dan 3 LED kanan bergantian
- Untuk membuat pola LED menyala secara bergantian antara sisi kiri dan kanan, digunakan pengelompokan pin dan perulangan *for* untuk mengontrol masing-masing kelompok LED. Pendekatan ini menunjukkan pemanfaatan struktur perulangan untuk mengatur beberapa output secara efisien dalam satu program [1], [4].

Pertanyaan Analisis

1. Uraikan hasil tugas pada praktikum yang telah dilakukan pada setiap percobaan
- Berdasarkan praktikum yang telah dilakukan, terdapat dua percobaan utama yaitu percobaan percabangan dan perulangan. Pada percobaan percabangan, LED menunjukkan perubahan kecepatan kedipan yang bersifat dinamis. Awalnya LED berkedip dengan interval yang lambat, kemudian semakin cepat seiring berkurangnya nilai *delay*. Ketika nilai mencapai batas tertentu, sistem melakukan reset sehingga pola kembali ke kondisi awal. Hal ini menunjukkan bahwa struktur *if-else* mampu mengatur alur program berdasarkan kondisi tertentu [4], [5].

Pada percobaan perulangan, dihasilkan pola *running LED* yang bergerak dari kiri ke kanan dan kembali ke kiri secara berulang. Pola ini terbentuk melalui penggunaan perulangan *for* yang mengatur urutan aktivasi pin digital secara sistematis. Hasil ini menunjukkan bahwa perulangan memungkinkan pengendalian banyak output secara efisien tanpa penulisan kode yang

berulang [1], [2], [3]. Secara keseluruhan, kedua percobaan menunjukkan hubungan langsung antara logika program dan respon perangkat keras pada sistem mikrokontroler [1].

2. Bagaimana pengaruh penggunaan struktur perulangan (seperti *for* dan *while*) terhadap jalannya program pada Arduino?
- Penggunaan struktur perulangan seperti *for* dan *while* memberikan pengaruh besar terhadap efisiensi dan keteraturan jalannya program. Perulangan memungkinkan eksekusi instruksi secara berulang tanpa harus menuliskan kode yang sama berkali-kali, sehingga program menjadi lebih ringkas dan mudah dikembangkan [1], [2], [3].

Dalam konteks Arduino, perulangan sangat berguna untuk mengontrol beberapa perangkat output seperti LED secara berurutan. Misalnya, pada percobaan *running LED*, perulangan *for* digunakan untuk menyalakan LED berdasarkan urutan pin sehingga menghasilkan efek visual tertentu. Selain itu, perulangan juga membantu dalam sinkronisasi waktu dan pengulangan proses secara konsisten dalam sistem embedded [4], [5].

4. Bagaimana cara kerja percabangan (*if-else*) dalam menentukan kondisi LED menyala atau mati berdasarkan input yang diberikan?
- Struktur percabangan *if-else* bekerja dengan cara mengevaluasi suatu kondisi logika yang bernilai benar atau salah. Jika kondisi bernilai benar (*true*), maka blok kode pada bagian *if* akan dijalankan. Sebaliknya, jika kondisi bernilai salah (*false*), maka program akan menjalankan blok kode pada bagian *else* [4], [5].

Dalam pengendalian LED, percabangan digunakan untuk menentukan kapan LED harus menyala atau mati. Jika suatu kondisi terpenuhi, Arduino akan memberikan sinyal HIGH untuk menyalakan LED. Jika tidak

terpenuhi, maka sinyal LOW diberikan untuk mematikan LED. Mekanisme ini memungkinkan sistem merespons perubahan kondisi secara dinamis [4].

3. Bagaimana pengaruh penggunaan struktur perulangan (seperti *for* dan *while*) terhadap jalannya program pada Arduino?

Kombinasi antara struktur perulangan dan percabangan memungkinkan pembuatan sistem yang responsif dan adaptif terhadap perubahan kondisi. Perulangan digunakan untuk menjalankan program secara terus-menerus, sedangkan percabangan digunakan untuk mengambil keputusan berdasarkan kondisi yang terjadi pada setiap siklus program [1], [2], [3].

Dalam sistem Arduino, fungsi *loop()* memastikan program berjalan secara kontinu. Di dalamnya, percabangan *if-else* digunakan untuk mengevaluasi kondisi tertentu, seperti nilai sensor atau variabel, sehingga sistem dapat menentukan tindakan yang sesuai. Dengan cara ini, Arduino dapat merespons perubahan lingkungan secara real-time, misalnya dalam sistem lampu otomatis berbasis sensor [4], [5].

Kombinasi kedua struktur ini merupakan dasar dalam pengembangan *embedded system*, karena memungkinkan sistem tidak hanya berjalan berulang tetapi juga mampu beradaptasi terhadap kondisi yang berubah selama operasi berlangsung [5].

5. KESIMPULAN

Berdasarkan praktikum Sistem Mikrokontroler Modul 1 yang telah dilakukan, dapat disimpulkan bahwa penerapan struktur kontrol percabangan dan perulangan pada Arduino Uno memiliki peran penting dalam mengendalikan perilaku *embedded system*. Melalui percobaan yang dilakukan, konsep dasar pemrograman dapat diimplementasikan secara langsung pada

perangkat keras sehingga menghasilkan respon visual berupa nyala LED [1], [2], [3].

Pada percobaan percabangan, penggunaan struktur *if-else* terbukti mampu mengatur perubahan kecepatan kedipan LED berdasarkan kondisi tertentu. Perubahan nilai variabel *timeDelay* secara bertahap menunjukkan bahwa logika percabangan memungkinkan sistem mengambil keputusan secara dinamis. Hal ini memperlihatkan bahwa mikrokontroler tidak hanya menjalankan instruksi secara linear, tetapi juga dapat merespons kondisi yang berubah selama program berjalan [4], [5].

Pada percobaan perulangan, penggunaan struktur *for* memungkinkan pengendalian beberapa LED secara berurutan dengan lebih efisien. Pola *running LED* yang dihasilkan menunjukkan bahwa perulangan dapat digunakan untuk mengatur urutan eksekusi secara sistematis, baik dari kiri ke kanan maupun sebaliknya. Hal ini membuktikan bahwa perulangan sangat efektif dalam menangani proses yang bersifat repetitif tanpa penulisan kode yang berulang [1], [2], [3].

Selain itu, praktikum ini juga memberikan pemahaman mengenai pentingnya penggunaan komponen pendukung seperti LED dan resistor dalam rangkaian elektronika. Penggunaan resistor sebagai pembatas arus menjadi aspek penting untuk menjaga keamanan komponen dan kestabilan sistem [3], [5].

Secara keseluruhan, praktikum ini berhasil memberikan pemahaman dasar mengenai hubungan antara logika program dan kerja perangkat keras pada sistem berbasis mikrokontroler. Dengan pemahaman ini, praktikan diharapkan mampu mengembangkan program yang lebih kompleks serta merancang *embedded system* yang lebih efektif dan efisien di masa mendatang.

DAFTAR PUSTAKA

- [1] Zulhelmi, D. Marpaung, dan S. Rejeki, *Penerapan Sistem Monitoring Penggunaan Listrik Berbasis IoT Pada Gedung Rektorat Universitas Lancang Kuning*, Dinamisia: Jurnal Pengabdian Kepada Masyarakat, Pekanbaru, 2022.
- [2] S. Samsugi, A. Ardiansyah, dan D. Kastutara, *Sistem Kendali Perangkat Elektronik Rumah Tangga Berbasis Android*, ILKOM Jurnal Ilmiah, Makassar, 2018.

- [3] A. B. Pulungan, A. S. Anwar, dan H. Hamdani, *Design of Smart Greenhouse Monitoring System based on Internet of Things*, SPEKTRA: Jurnal Fisika dan Aplikasinya, Jakarta, 2018.
- [4] <https://www.arduinoindonesia.id/2024/05/kontrol-flow-dalam-pemrograman-arduino-if-else-dan-loop.html>, 8 April 2026, 23:17 WIB.
- [5] <https://www.elcreativeacademy.com/2019/11/dasar-dasar-arduino-fungsi-logika.html>, 8 April 2026, 23:17 WIB.